Investigación del sistema de reservas por seis grupos

Marco teórico aplicado al caso de éxito propio

La investigación se organiza en seis grupos y vincula sus hipótesis con el sistema de reservas construido, las evidencias locales y el experimento OpenShift pendiente.

Índice

Actualizar el índice en Word con F9.

Propósito y caso de éxito propio

El caso de éxito de nuestro seminario es el sistema de reservas que construimos. Permite consultar eventos y cupos, crear una reserva, recuperar una operación cuya respuesta no se recibió, consultar su comprobante y salir de la sesión. La investigación explica cómo se protege esa operación y qué necesita la plataforma para sostenerla al crecer la demanda. Cada concepto se relaciona con un componente, una prueba y una evidencia del mismo sistema.

Los archivos organizacion_grupos_seminario.docx y caso_exito_kubernetes_openshift.docx aportados por el responsable establecen seis grupos, 22 alumnos y el horizonte de 1 a 10 millones de transacciones. Esta edición conserva esos temas y objetivos y los adapta a lo construido. Las hipótesis son preguntas por contrastar; no se convierten en resultados por llamar caso de éxito al proyecto.

El logro acreditado es funcional y experimental en laboratorio local: aplicación integrada, inventario protegido, idempotencia y mediciones reproducibles en Compose. No se ha demostrado sostener 10 millones de reservas en OpenShift, disponibilidad de 99.9 por ciento, autoescalado, Kafka ni detección de degradación con precisión del 95 por ciento. Mostraremos el sistema y sus evidencias y explicaremos el experimento que los seis equipos deben integrar para comprobar esas metas.

DOC-01-01, organizado en 22 capítulos generales, queda como biblioteca técnica complementaria; esta investigación es el recorrido principal por grupos. Una copia de la edición anterior se conserva en documentacion/fase-01/historico/v1-2026-10-13. Los dos Word originales del usuario no se modifican.

Organización de la investigación

Grupo y alumnos	Tema asignado	Aporte al sistema de reservas
G1 · 4	Fundamentos y Arquitectura Kubernetes y OpenShift	Componentes fronteras y arquitectura de despliegue
G2 · 4	Escalabilidad y Autoescalado	Crecimiento de API y procesamiento asíncrono
G3 · 4	Resiliencia Disponibilidad y Seguridad	Fallos recuperación acceso y restricciones de clúster
G4 · 4	Persistencia Consistencia y Mensajería	Cupos reservas únicas y propagación de eventos
G5 · 3	Observabilidad y Métricas	Visibilidad de negocio y plataforma

Grupo y alumnos	Tema asignado	Aporte al sistema de reservas
G6 · 3	Diseño Experimental y Análisis Estadístico	Protocolo ejecución comparación y conclusión

Los seis grupos investigan una sola operación de negocio. G1 fija la arquitectura; G4 define las garantías del dato; G2 y G3 establecen crecimiento y continuidad; G5 prepara instrumentos; G6 integra tratamientos y comprueba que la conclusión corresponda a los datos. Cada representante entrega semanalmente a G6 versiones, configuración, hallazgos, limitaciones y evidencia. La integración no se deja para el día de exposición.

El sistema de reservas como hilo conductor

Operación y alcance

El participante ingresa su clave temporal, consulta catálogo y disponibilidad, elige una cantidad y confirma. La API comprueba identidad y contenido y solicita una escritura transaccional a PostgreSQL. Si hay cupo, reserva, descuento y evento outbox se guardan en la misma transacción. Un comprobante identifica la reserva confirmada. El procesador de outbox puede registrar una recepción simulada en la base; no se implementó correo externo.

Una reserva asigna cupos de un evento. No implementamos pagos, facturación ni selección numerada de asientos. La documentación usa siempre sistema de reservas. El frontend mantiene la clave solo en memoria y conserva la intención original para recuperar una operación incierta: perder la respuesta no significa que la base rechazó la reserva.

Arquitectura existente y ampliación propuesta

La confirmación actual es síncrona respecto de PostgreSQL. Un 201 significa reserva confirmada; un 200 de reenvío devuelve la misma reserva. Responder por haber encolado una solicitud en Kafka cambiaría ese contrato. Para incorporar mensajería sin presentar otro sistema, proponemos publicar desde outbox después del commit y estudiar KEDA sobre consumidores asíncronos. Una recepción 202 con consulta de estado exigiría un diseño y pruebas adicionales.

La topología medida en S19 contiene Nginx, una API, PostgreSQL y un generador HTTP en Compose, sin worker ni broker. Los manifiestos preparados para kind y OpenShift sirven para explicar objetos y restricciones, pero todavía no acreditan admisión del clúster. La arquitectura local comprobada y la futura deben mostrarse como vistas distintas.

Grupo 1 Fundamentos y Arquitectura

Fundamento y problema de investigación

G1 explica qué aporta Kubernetes a nuestro sistema y qué sigue siendo responsabilidad de la aplicación. La imagen empaqueta archivos y dependencias; el contenedor ejecuta ese contenido con aislamiento. Kubernetes declara estado deseado mediante objetos API y usa controladores para

reconciliarlo. El plano de control recibe la configuración y decide; los nodos ejecutan mediante kubelet y un runtime. Una réplica nueva requiere recursos y dependencias; reemplazar un Pod no recupera por sí mismo una transacción de reserva. [R01]

Deployment mantiene frontend o API; Service ofrece acceso estable; ConfigMap y Secret separan configuración y datos sensibles; PVC solicita almacenamiento. StatefulSet organiza identidad y volúmenes de cargas con estado, pero no configura automáticamente replicación, recuperación ni alta disponibilidad de PostgreSQL. El estado del clúster en etcd no sustituye la base de reservas.

OpenShift es la plataforma empresarial elegida por el planteamiento de investigación. Route, SCC, Operators y herramientas de entrega deben analizarse por su efecto concreto en reservas. Kubernetes upstream y OKD son contexto comparativo, no otros casos de éxito. La investigación debe registrar versión, mecanismo, decisión y prueba, sin convertirse nuevamente en un catálogo general de productos. [R02]

Aplicación e hipótesis

La arquitectura separa navegador, frontend estático con proxy, API, PostgreSQL y procesador de outbox. La API no guarda el inventario definitivo en memoria del proceso; eso facilita replicarla. Sin embargo, conexiones, bloqueos y capacidad de datos siguen limitando el conjunto. Tener componentes separados no demuestra superioridad respecto de un monolito equivalente.

La hipótesis original de G1 compara microservicios stateless con un monolito. Hace falta un comparador con la misma funcionalidad, datos y recursos para contrastarla. Hoy existe un backend modular con API y worker; no hay una comparación monolito contra microservicios ejecutada. G1 debe estudiar ese fundamento y diseñar la comparación sin adjudicarle resultados.

Entregable y exposición

G1 aporta las secciones 2.1 y la base arquitectónica de 2.2 del documento rector, inventario de componentes, contratos, diagrama editable y ficha de despliegue. En la exposición sigue una solicitud real desde navegador hasta base y diferencia contenedor, Pod y Service. Entrega a G6 imágenes identificadas, réplicas, nodos, recursos y dependencias por condición experimental.

Grupo 2 Escalabilidad y Autoescalado

Mecanismos y señales

Escalar horizontalmente la API reparte solicitudes entre instancias, si hay capacidad de nodos y datos. HPA ajusta réplicas con métricas y una política de control; no amplía PostgreSQL. Cuando utiliza CPU relativa a requests, cambiar esos requests modifica el significado del porcentaje objetivo. Los ensayos deben conservar esa configuración o declarar su variación. [R03]

VPA estudia el tamaño de recursos de los Pods; Cluster Autoscaler trata capacidad de nodos según la infraestructura disponible. No resuelven lo mismo que HPA. La primera comparación propone VPA en recomendación y recursos fijos, para no variar tamaño y cantidad de réplicas al mismo tiempo. El autoescalado de nodos será otro tratamiento si existe infraestructura compatible. Una laptop no crea recursos físicos por añadir nodos virtuales. [R04]

KEDA es pertinente para procesadores asíncronos. Con Kafka, el rezago de consumidor puede orientar workers mientras HPA atiende la API. No se crean controladores independientes que compitan por las mismas réplicas. El paralelismo útil de un grupo consumidor depende de particiones, asignación y capacidad de la base: más workers no garantizan menos rezago. [R05]

Hipótesis y contraste

La hipótesis original propone throughput proporcional al volumen hasta 10M sin crecimiento desproporcionado de p95. Antes se deben separar volumen N, tasa ofrecida y duración: un millón distribuido en horas no equivale a un millón en minutos. Se medirán tasas ofrecida y completada, latencias, errores, réplicas y colas, además del tiempo de reacción y estabilización.

Primero se compara capacidad fija contra HPA de API. Después se agrega el consumidor Kafka y KEDA con su propia línea base. VPA, malla y autoescalado de nodos se estudian aparte. Activarlo todo a la vez impediría identificar qué produjo una diferencia. Debe distinguirse un beneficio por mejor control de otro por asignar un presupuesto mayor de recursos.

S19 es una línea base de consumo, no de escalado: hubo una API, cuatro usuarios y carga cerrada próxima a 35.95 HTTP/s, sin buscar saturación. La limitación de CPU observada permite formular una hipótesis, pero no demuestra que HPA la resuelva ni que PostgreSQL soporte el siguiente nivel.

Entregable y exposición

G2 entrega el estudio de 2.2, señales, políticas y configuraciones revisables de HPA y KEDA, prerrequisitos de métricas y límites de escalado. En la exposición distingue crecimiento por solicitudes de reserva de crecimiento por eventos pendientes. Una animación de Pods se rotula propuesta hasta disponer de la serie real del clúster.

Grupo 3 Resiliencia Disponibilidad y Seguridad

Fallos y recuperación

Una interfaz disponible no significa que se pueda reservar. El proxy puede responder mientras la base falla; una reserva puede persistir aunque su respuesta se pierda. G3 estudia frontend, API, base, worker, red y nodo por separado y define efecto visible, integridad y recuperación de cada fallo.

Startup, readiness y liveness permiten evaluar arranque, envío de tráfico y necesidad de reiniciar procesos. Acoplar una sonda de vida a una dependencia caída puede provocar reinicios innecesarios. En nuestro frontend readiness verifica el recurso estático, no PostgreSQL. S18 detuvo la API exclusiva, obtuvo un 502 genérico y comprobó recuperación: evidencia local de ese fallo, no de caída de nodo OpenShift. [R06]

PodDisruptionBudget restringe determinadas interrupciones voluntarias, como desalojos de mantenimiento. No impide todos los fallos ni toda eliminación de Pods. Disponibilidad requiere réplicas, distribución, capacidad y continuidad del dato. Una sola instancia PostgreSQL continúa siendo un punto de fallo aunque haya varias API. [R07]

Acceso SCC y Service Mesh

SCC controla permisos admitidos en OpenShift. La imagen frontend pasó UID arbitrario, raíz de solo lectura, capacidades eliminadas y temporales en /tmp sobre Docker. La admisión SCC real sigue pendiente. El grupo debe registrar SCC aplicada, UID efectivo, cuenta de servicio y eventos; anyuid no es una solución general sin justificación. [R08]

La clave temporal se entrega por un canal privado y no se guarda en URL, bundle o evidencias. La autorización de consulta también reside en servidor: conocer un UUID no debe revelar la reserva ajena. RBAC, namespaces, políticas de red, certificados y rotación se plantean como controles con pruebas, no como una lista de herramientas.

Service Mesh puede aportar gestión de tráfico, seguridad y telemetría. Sus reintentos requieren respetar idempotencia: la malla no vuelve atómica una reserva ni garantiza efectos externos exactamente una vez. Debe estudiarse sobrecarga y comparar condiciones con y sin malla. Esa comparación todavía no se ejecutó. [R09]

Hipótesis y entregable

La disponibilidad superior a 99.9 por ciento permanece como hipótesis. Exige ventana, denominador y clasificación; se separa disponibilidad temporal de proporción de solicitudes elegibles exitosas. Una demostración breve sin errores no acredita disponibilidad mensual. G3 entrega 2.3 y la parte SCC y malla de 2.5, matriz de fallos, controles y protocolo de recuperación. Durante la demostración explica cómo recuperar una respuesta perdida sin duplicar el descuento.

Grupo 4 Persistencia Consistencia y Mensajería

Inventario transacciones e idempotencia

La garantía central es no confirmar más cupos de los existentes. Leer disponibilidad no bloquea cupos: dos participantes pueden ver el mismo valor. La decisión definitiva ocurre mediante escritura condicional y transacción en PostgreSQL. Reserva, descuento y outbox forman una unidad que debe confirmarse o revertirse completa. Las restricciones y bloqueos protegen el dato frente a clientes concurrentes. [R10]

La clave de idempotencia distingue intención de intento HTTP. Mismo propietario, clave y contenido devuelven la reserva original; contenido distinto produce conflicto. Impedir doble clic no controla reintentos, clientes paralelos o respuestas perdidas. El comprobante permite consultar después, pero no sustituye la autenticación.

ACID describe propiedades de transacciones, no crecimiento lineal de toda arquitectura. CAP trata las garantías de consistencia y disponibilidad cuando ocurre una partición bajo sus definiciones; no obliga a abandonar ACID por agregar réplicas. En reservas se prioriza no confirmar inventario inexistente, aunque una dependencia no disponible obligue a responder error o a conservar una intención incierta. [R11]

Outbox y Kafka aplicados al sistema

Outbox evita confirmar la reserva y perder su evento antes de publicarlo. El worker puede repetir el procesamiento; el receptor necesita deduplicación. Aquí la recepción simulada se registra de forma

única en PostgreSQL. Esa prueba no acredita entrega exactamente una vez a correo o sistemas externos. Debe indicarse qué paso confirma cada componente y qué se repite tras reiniciar.

La ampliación propuesta publica eventos de reservas confirmadas desde outbox hacia Kafka o AMQ Streams. Cada mensaje define identificadores de evento y reserva, tipo y versión de esquema, sin la clave privada. La partición se decide según el orden requerido. Usar event_id puede conservar orden para un evento, pero también concentrar un evento muy demandado en una sola partición. Kafka no reemplaza la protección transaccional del inventario. [R12]

G4 debe especificar deduplicación del consumidor, retención, compatibilidad de esquema, mensajes inválidos y pruebas de caída. Una réplica de lectura potencialmente retrasada no debe autorizar un descuento de cupos como si fuera la escritura definitiva. El procesamiento eventual de notificaciones y la confirmación de reserva son garantías diferentes.

Hipótesis y entregable

Se conserva la hipótesis de escritura final consistente y propagación asíncrona sin pérdida ni duplicación de efectos. G4 entrega 2.4 y 5.2, modelo de datos, invariantes, topic y particionado y matriz de deduplicación. En la exposición muestra creación y reenvío con el mismo identificador y un único descuento. S19 auditó 380 reservas y 380 outbox sin huérfanos, pero no tenía worker; no acredita notificaciones procesadas.

Grupo 5 Observabilidad y Métricas

Medir plataforma y negocio

G5 debe permitir responder si se puede reservar, cuánto tarda, cuántas reservas únicas se confirmaron y qué componente limita el servicio. CPU sola no responde esas preguntas. Se correlacionan cliente, plataforma y negocio con marcas de tiempo y run_id. OpenTelemetry diferencia métricas, logs y trazas; tener una dependencia instalada no equivale a instrumentar y recolectar esas señales. [R13]

El tablero propuesto muestra tasa ofrecida y completada, p95 y p99 por operación, errores técnicos, rechazos de negocio, reservas únicas, cupos, conflictos de idempotencia, rezago, réplicas listas, CPU, memoria, cuotas y OOM. Las etiquetas deben ser acotadas; no se usan claves privadas ni un UUID distinto por reserva como etiquetas de series.

Instrumentación existente y pendiente

La API expone contadores y suma y conteo de duración por ruta. Con ese par puede calcularse una media, no reconstruirse p95 ni p99. Para percentiles se necesita una distribución adecuada, por ejemplo histogramas, o latencias individuales del cliente. Prometheus y Grafana requieren configuración y datos reales; un diseño de dashboard no se presenta como uno ya operativo. S19 utilizó observaciones crudas, Docker stats y cgroup v2, no un stack completo de Prometheus, Grafana y Jaeger. [R14]

Las trazas propuestas siguen entrada, validación, acceso a datos y publicación o consumo. Se deben definir propagación, muestreo, retención y eliminación de secretos, y explicar cómo el muestreo limita las conclusiones. La trazabilidad tampoco sustituye una auditoría de inventario y reservas.

Hipótesis de detección y entregable

La precisión objetivo del 95 por ciento requiere una verdad de referencia. Se propone etiquetar ventanas de degradación según un SLO congelado y contar verdaderos positivos, falsos positivos, falsos negativos y demora de detección. Precisión es TP dividido entre TP más FP; no es sensibilidad, exactitud ni un intervalo de confianza del 95 por ciento. No se promete localizar un punto exacto de degradación con un muestreo que no lo permite.

G5 entrega dashboards, consultas, alertas, diccionario de métricas y configuración propuesta de Prometheus Adapter para G2. Cada panel debe indicar fuente y unidad. En la exposición muestra resultados S19 como evidencia actual y el tablero del experimento OpenShift como propuesta. Entrega a G6 series crudas y eventos, no solamente capturas.

Grupo 6 Diseño Experimental y Análisis Estadístico

Pregunta y unidad de análisis

La pregunta es si el sistema de reservas sobre OpenShift conserva objetivos de latencia, disponibilidad e integridad al aumentar la carga con políticas seleccionadas de escalado. Definimos transacción lógica como un intento de creación con una clave de idempotencia distinta. Reenvíos, consultas y assets son HTTP adicionales; no agregan intenciones nuevas. Confirmaciones y rechazos de negocio se cuentan por separado.

S19 midió 3240 HTTP, pero creó 360 reservas lógicas nuevas durante las repeticiones y veinte en calentamiento. No se puede llamar transacciones de reserva a los 3240 ni extrapolarlos a un millón. Un ensayo de N confirmaciones requiere inventario suficiente, estado inicial definido y espacio para reservas, índices, outbox y telemetría.

Niveles condiciones y recursos

Se conservan 1M, 2.5M, 5M, 7.5M y 10M de intenciones únicas por corrida. Además de N, cada nivel fija tasa, duración máxima, perfil y regla de parada. N dividido entre duración es solo un promedio y no describe las ráfagas. Primero se valida generador e instrumentación con un piloto menor; no se supone que una laptop pueda ejecutar toda la escala.

La primera comparación es capacidad fija frente a HPA de API, con al menos tres corridas por nivel y condición y estado inicial equivalente. El orden se aleatoriza o bloquea para reducir efectos de entorno y calentamiento. Kafka y KEDA necesitan otra etapa y su propia línea base; VPA, malla y nodos no cambian simultáneamente sin control. Los cinco niveles suman 26M; dos condiciones y tres repeticiones representan 156M de intenciones, antes de reenvíos y lecturas. G6 debe validar costo, almacenamiento y capacidad del generador antes de ejecutar ese alcance.

Criterios previos y análisis

Se proponen para discusión p95 de creación menor o igual a 800 ms, error técnico menor a 1 por ciento y disponibilidad de solicitudes elegibles de al menos 99.9 por ciento en una ventana definida. Son umbrales propuestos, no un SLA contractual ni resultados obtenidos. El responsable y G6 deben congelar además p99, calentamiento, datos faltantes y precisión de instrumentos. La integridad exige cero sobreventa, reserva única por intención y outbox coherente.

Se registran por separado confirmaciones por segundo y HTTP por segundo, retraso de escalado, réplicas listas, colas y consumo. Un rechazo por inventario insuficiente no se clasifica igual que un timeout; tampoco cuenta como reserva confirmada. El caso de inventario agotado puede probarse funcionalmente sin mezclarlo con la medición de capacidad de confirmaciones.

La corrida, no cada solicitud correlacionada, es la unidad principal de comparación. Se conservan distribuciones y percentiles por corrida; promediar tres p95 no produce un p95 conjunto. El análisis debe corresponder al diseño: corridas emparejadas por bloque no se tratan automáticamente como muestras independientes. Tres repeticiones son un mínimo exploratorio, no garantía de potencia estadística. [R15]

Se estiman efectos e incertidumbre y se compara con umbrales. No detectar diferencia no demuestra equivalencia. Un resultado favorable en 10M no acredita todo el rango si otro nivel falló. Las proporciones cercanas a uno requieren intervalos apropiados y consideración de dependencia temporal; una secuencia breve sin errores no prueba una disponibilidad poblacional exacta. [R16]

G6 entrega protocolo previo, configuración, matriz de corridas, datos y análisis reproducible y conclusión que distinga hipótesis apoyada, refutada o sin evidencia suficiente. H0 y H1 del documento rector se conservan como formulación de partida y se operacionalizan por indicador. Hoy existe una base funcional y un piloto local; el contraste de 1M a 10M en OpenShift sigue pendiente.

Plan de exposición y demostración conjunta

Se abre con la necesidad de reservar sin sobreventa ni duplicación. Se presentan los seis grupos y cada equipo explica su mecanismo sobre el mismo caso. La demostración integrada dura 25 minutos propuestos: G1 ubica componentes; G3 explica ingreso seguro; G4 acompaña creación y comprobante; G2 indica qué se escalaría; G5 observa indicadores; G6 verifica e interpreta el resultado.

Se muestran acceso, catálogo, disponibilidad, reserva de una unidad, confirmación, copia, consulta y salida. Se anota disponibilidad inicial y final y se conserva el identificador. Idempotencia y concurrencia se explican con pruebas aisladas o evidencias existentes; no se inventa un doble clic que la interfaz bloquea ni se altera la base medida de S19. Simular una respuesta perdida exige un entorno de ensayo dedicado y no una interrupción improvisada frente al público.

La presentación DOC-06-02 sigue los seis grupos y la guía DOC-07-02 detalla preparación, responsables y aceptación. Si no hay clúster, los manifiestos se muestran como propuesta; no se afirma que OpenShift escaló porque Compose sirvió una reserva.

Matriz de evidencia y aceptación

Evidencia o meta	Estado actual	Uso académico
Frontend API y reserva transaccional	Implementados y probados localmente	Demostración del caso propio

Evidencia o meta	Estado actual	Uso académico
Idempotencia e inventario	Pruebas y auditorías	Sustento de G4 y G6
Imagen proxy y UID arbitrario	Verificados en Compose S18	G1 y G3 sin afirmar SCC
Tres corridas S19	3240 HTTP y 360 reservas nuevas medidas	Línea base de G5 y G6
Clúster HTTPS SCC HPA	Preparación y pruebas pendientes	Trabajo de plataforma
Kafka KEDA y trazas	Ampliaciones propuestas	G2 G4 y G5
1M a 10M y 99.9 por ciento	Hipótesis sin contraste a esa escala	Experimento integrado de G6

El valor académico del caso propio está en la aplicación construida, sus garantías verificadas y un protocolo que permite comprobar o refutar objetivos. Cada equipo aporta fuentes, decisiones, artefactos y evidencia con procedencia. No se atribuyen a alumnos específicos trabajos o resultados sin sus registros. Esta adaptación documental no implementa servicios nuevos ni ejecuta el experimento de millones de reservas.

Referencias y procedencia

[U1] Responsable del seminario. organizacion_grupos_seminario.docx. Seis grupos y 22 alumnos; fecha interna agosto de 2026. Original conservado sin modificación.

[U2] Responsable del seminario. caso_exito_kubernetes_openshift.docx. Planteamiento de problema marco teórico e hipótesis de 1M a 10M; propuesta de investigación, no reporte de resultados ejecutados.

[E1] Fuentes y pruebas del proyecto: aplicacion/reservas, aplicacion/tests y evidencias históricas de Fases 04 y 05. Cada medición conserva su topología.

[E2] S18: documentacion/fase-10/sprints/S18 y evidencias/frontend-s18. Commit local d7fea11; clúster pendiente.

[E3] S19: documentacion/fase-10/sprints/S19 y evidencias/frontend-s19. summary.json y database-audit.json; entrega en revisión al preparar esta adaptación.

[R01] Kubernetes. Overview y Deployment. https://kubernetes.io/docs/concepts/overview/ y https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

[R02] Red Hat. OpenShift Container Platform. https://docs.redhat.com/en/documentation/openshift_container_platform/

[R03] Kubernetes. Horizontal Pod Autoscaling. https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

[R04] Kubernetes. Vertical Pod Autoscaling y gestión de recursos. https://kubernetes.io/docs/concepts/workloads/autoscaling/vertical-pod-autoscale/ y https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/

[R05] KEDA. Apache Kafka scaler. https://keda.sh/docs/2.21/scalers/apache-kafka/ Fijar versión instalada antes de usar como configuración.

[R06] Kubernetes. Probes. https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-probes/

[R07] Kubernetes. Disruptions. https://kubernetes.io/docs/concepts/workloads/pods/disruptions/

[R08] Red Hat. Security context constraints. https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/authentication_and_authorization/managing-pod-security-policies

[R09] Red Hat. OpenShift Service Mesh. https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/

[R10] PostgreSQL. Transaction isolation. https://www.postgresql.org/docs/current/transaction-iso.html

[R11] Gilbert y Lynch. Brewer's conjecture and the feasibility of consistent available partition tolerant web services. ACM SIGACT News 2002. https://doi.org/10.1145/564585.564601

[R12] Apache Kafka. Design. https://kafka.apache.org/documentation/#design

[R13] OpenTelemetry. Signals. https://opentelemetry.io/docs/concepts/signals/

[R14] Prometheus. Histograms and summaries. https://prometheus.io/docs/practices/histograms/

[R15] NIST. Analysis of paired observations. https://www.itl.nist.gov/div898/handbook/prc/section3/prc311.htm

[R16] NIST. Confidence intervals for proportions. https://itl.nist.gov/div898/handbook/prc/section2/prc241.htm

Consulta técnica de la adaptación: 20 de octubre de 2026. La existencia de una función en documentación de producto no acredita instalación ni desempeño en nuestro entorno.